

Язык программирования C++

Лекция 11: временные объекты, new/delete с объектами, умные указатели, наследование

Никита Лисица

2026

- Из-за присутствия конструкторов и деструкторов, в C++ приходится внимательнее следить за временем жизни объектов

Временные объекты

- Из-за отсутствия конструкторов и деструкторов, в C++ приходится внимательнее следить за временем жизни объектов

```
int max(int a, int b) {  
    return a < b ? b : a;  
}  
  
int main() {  
    // `a` живёт от этой строки и до конца main  
    int a = 2;  
  
    // `b` живёт от этой строки и до конца main  
    int b = 3;  
  
    max(a, b);  
}
```

- Что происходит, если написать `max(2, 3)`?

- Что происходит, если написать `max(2, 3)`? Создаются временные объекты:

Временные объекты

- Что происходит, если написать `max(2, 3)`? Создаются временные объекты:

```
int max(int a, int b) {  
    return a < b ? b : a;  
}  
  
int main() {  
    // Здесь, `2` и `3` это временные объекты  
    // Они создаются перед вызовом функции, и  
    // живут до окончания выполнения всего выражения  
    // Возвращаемое значение - тоже временный объект  
    max(2, 3);  
}
```

- Это не очень важно для встроенных типов, но важно для классов с нетривиальными конструкторами:

Временные объекты

- Это не очень важно для встроенных типов, но важно для классов с нетривиальными конструкторами:

```
BigInt max(const BigInt& a, const BigInt& b) {  
    return a < b ? b : a;  
}  
  
int main() {  
    // В этой строке создаются два временных объекта  
    // типа BigInt  
    max(  
        BigInt("12347987955"),  
        BigInt("867577959256")  
    );  
}
```


Параметры по умолчанию

- У функций можно указывать параметры по умолчанию, тогда их не обязательно передавать:

```
void startServer(const char* name, int port = 8080,  
                const char* host = "0.0.0.0");  
  
int main()  
{  
    startServer("minecraft", 25565, "127.0.0.1");  
    startServer("postgresql", 5432);  
    startServer("ping");  
}
```

- Они могут быть только последними параметрами (иначе непонятно, какие параметры передали)

- Они могут быть только последними параметрами (иначе непонятно, какие параметры передали)
- Вместе с перегрузкой функций может приводить к неожиданным эффектам или к неопределённости

Параметры по умолчанию

- Они могут быть только последними параметрами (иначе непонятно, какие параметры передали)
- Вместе с перегрузкой функций может приводить к неожиданным эффектам или к неопределённости
- Параметры по умолчанию подставляются компилятором в код вызова функции $\implies$ должны быть описаны в объявлении функции (обычно в header-файле)

- Пусть, у нас есть класс

```
class Server {  
public:  
    Server(int port);  
    ~Server();  
};
```

- Пусть, у нас есть класс

```
class Server {  
public:  
    Server(int port);  
    ~Server();  
};
```

- Как нам создать объект такого класса в куче?

- Пусть, у нас есть класс

```
class Server {  
public:  
    Server(int port);  
    ~Server();  
};
```

- Как нам создать объект такого класса в куче?
- `malloc` позволит выделить память, но не вызовет конструктор

- Пусть, у нас есть класс

```
class Server {  
public:  
    Server(int port);  
    ~Server();  
};
```

- Как нам создать объект такого класса в куче?
- `malloc` позволит выделить память, но не вызовет конструктор
- `free` не вызовет деструктор

new/delete с объектами

- Решение: **new/delete!**

```
// Можем передать параметры конструктора
Server * server = new Server(5432);
...
// Вызывает деструктор
delete server;
```

new/delete с объектами

- Решение: **new/delete!**

```
// Можем передать параметры конструктора
Server * server = new Server(5432);
...
// Вызывает деструктор
delete server;
```

- Почему компилятор сам не вставит вызов деструктора?

new/delete с объектами

- Решение: **new/delete!**

```
// Можем передать параметры конструктора
Server * server = new Server(5432);
...
// Вызывает деструктор
delete server;
```

- Почему компилятор сам не вставит вызов деструктора? Он не знает, какое время жизни мы хотим у объекта ***server**

new/delete с объектами

- Решение: **new/delete**!

```
// Можем передать параметры конструктора
Server * server = new Server(5432);
...
// Вызывает деструктор
delete server;
```

- Почему компилятор сам не вставит вызов деструктора? Он не знает, какое время жизни мы хотим у объекта ***server**
- Он вызовет деструктор у самого указателя **server** – но деструкторы примитивных типов ничего не делают

Placement new

- Можно и через `malloc/free`, но руками вызвать конструктор и деструктор:

```
Server * server = malloc(sizeof(Server));

// Особый синтаксис вызова конструктора
// у объекта по указателю (не выделяет память)
// - т.н. placement new
new (server) Server(5432);
...

// Руками вызываем деструктор
server->~Server();

free(server);
```

- Обычно нужно только при реализации структур данных (напр. `std::vector`), в большинстве случаев хватает обычных `new/delete`

- Обычно нужно только при реализации структур данных (напр. `std::vector`), в большинстве случаев хватает обычных `new/delete`
- Даже если вы выделили объект через `malloc` + placement new, его нельзя удалять через `delete`, только через ручной вызов деструктора + `free`

- Аналогично, `new[]` не только выделит память, но и вызовет конструктор по умолчанию у всех объектов

`new[]/delete[]` с объектами

- Аналогично, `new[]` не только выделит память, но и вызовет конструктор по умолчанию у всех объектов
- `delete[]` пройдёт по всем объектам и вызовет у каждого деструктор

`new[]/delete[]` с объектами

- Аналогично, `new[]` не только выделит память, но и вызовет конструктор по умолчанию у всех объектов
- `delete[]` пройдёт по всем объектам и вызовет у каждого деструктор

```
// Вызовет конструктор BigInt() у элементов
// от bigInts[0] до bigInts[99]
BigInt * bigInts = new BigInt[100];

// Вызовет деструктор у элементов
// от bigInts[0] до bigInts[99]
delete[] bigInts;
```

`new[]/delete[]` с объектами

- Аналогично, `new[]` не только выделит память, но и вызовет конструктор по умолчанию у всех объектов
- `delete[]` пройдёт по всем объектам и вызовет у каждого деструктор

```
// Вызовет конструктор BigInt() у элементов
// от bigInts[0] до bigInts[99]
BigInt * bigInts = new BigInt[100];

// Вызовет деструктор у элементов
// от bigInts[0] до bigInts[99]
delete[] bigInts;
```

- А если нет конструктора по умолчанию?

new[]/delete[] с объектами

- Аналогично, `new[]` не только выделит память, но и вызовет конструктор по умолчанию у всех объектов
- `delete[]` пройдёт по всем объектам и вызовет у каждого деструктор

```
// Вызовет конструктор BigInt() у элементов
// от bigInts[0] до bigInts[99]
BigInt * bigInts = new BigInt[100];

// Вызовет деструктор у элементов
// от bigInts[0] до bigInts[99]
delete[] bigInts;
```

- А если нет конструктора по умолчанию? `malloc` + `placement new`

- Многовато вызовов `delete`! Воспользуемся инкапсуляцией и деструкторами $\Rightarrow$ *умные указатели*

Умные указатели

- Многовато вызовов `delete`! Воспользуемся инкапсуляцией и деструкторами $\Rightarrow$ *умные указатели*
- Без умных указателей:

```
Object* p = new Object(...);  
p->doSomething();  
// Надо не забыть вызвать delete  
delete p;
```

Умные указатели

- Многовато вызовов `delete`! Воспользуемся инкапсуляцией и деструкторами $\Rightarrow$ *умные указатели*
- Без умных указателей:

```
Object* p = new Object(...);  
p->doSomething();  
// Надо не забыть вызвать delete  
delete p;
```

- С умными указателями:

```
scoped_ptr p = new Object(...);  
p->doSomething();  
// деструктор scoped_ptr вызовет delete
```

scoped_ptr

```
class scoped_ptr {
private:
    Object* ptr_;

public:
    scoped_ptr(Object* ptr);
    ~scoped_ptr();

    Object* get() const;
    Object& operator * () const;
    Object* operator -> () const;
    explicit operator bool() const;

    // Запрет копирования после C++11:
    scoped_ptr(const scoped_ptr& other) = delete;
    const scoped_ptr& operator = (const scoped_ptr& p) = delete;

    // Запрет копирования до C++11:
private:
    scoped_ptr(const scoped_ptr& other);
    const scoped_ptr& operator = (const scoped_ptr& p);
};
```


scoped_ptr

```
scoped_ptr::scoped_ptr(Object* ptr)
    : ptr_(ptr)
{}

scoped_ptr::~~scoped_ptr() {
    delete ptr_;
}

Object* scoped_ptr::get() const { return ptr_; }

Object& scoped_ptr::operator * () const { return *ptr_; }

Object* scoped_ptr::operator -> () const { return ptr_; }

scoped_ptr::operator bool() const { return ptr_ != nullptr; }
```

- Зачем запрет копирования?

- Зачем запрет копирования? Чтобы два разных `scoped_ptr` не пытались удалить один и тот же указатель

- Зачем запрет копирования? Чтобы два разных `scoped_ptr` не пытались удалить один и тот же указатель
- Как работает `operator ->`?

- Зачем запрет копирования? Чтобы два разных `scoped_ptr` не пытались удалить один и тот же указатель
- Как работает `operator ->`? Возвращает либо указатель (у которого есть встроенный оператор), либо что-нибудь, у чего тоже есть `operator ->`, и вызывает его рекурсивно

- `scoped_ptr`: нельзя копировать, нельзя передать владение объектом (напр. в функцию или из функции), объект живёт ровно внутри одного `scope`, только один `scoped_ptr` указывает на объект

- `scoped_ptr`: нельзя копировать, нельзя передать владение объектом (напр. в функцию или из функции), объект живёт ровно внутри одного scope, только один `scoped_ptr` указывает на объект
- `unique_ptr`: нельзя копировать, можно перемещать (*move*, об этом позже), можно передавать владение, только один `unique_ptr` указывает на объект

- `scoped_ptr`: нельзя копировать, нельзя передать владение объектом (напр. в функцию или из функции), объект живёт ровно внутри одного `scope`, только один `scoped_ptr` указывает на объект
- `unique_ptr`: нельзя копировать, можно перемещать (*move*, об этом позже), можно передавать владение, только один `unique_ptr` указывает на объект
- `shared_ptr`: можно копировать ($\implies$ можно передавать владение), несколько `shared_ptr` могут указывать на один объект

- `scoped_ptr`: нельзя копировать, нельзя передать владение объектом (напр. в функцию или из функции), объект живёт ровно внутри одного `scope`, только один `scoped_ptr` указывает на объект
- `unique_ptr`: нельзя копировать, можно перемещать (*move*, об этом позже), можно передавать владение, только один `unique_ptr` указывает на объект
- `shared_ptr`: можно копировать ($\implies$ можно передавать владение), несколько `shared_ptr` могут указывать на один объект
- **NB:** На самом деле, умные указатели – это шаблоны классов, но о шаблонах мы поговорим позже

- Идея: храним счётчик ссылок

- Идея: храним счётчик ссылок
- Когда `shared_ptr` копируется, увеличиваем счётчик

- Идея: храним счётчик ссылок
- Когда `shared_ptr` копируется, увеличиваем счётчик
- Когда `shared_ptr` удаляется, уменьшаем счётчик

- Идея: храним счётчик ссылок
- Когда `shared_ptr` копируется, увеличиваем счётчик
- Когда `shared_ptr` удаляется, уменьшаем счётчик
- Когда счётчик доходит до нуля (т.е. удалили последний `shared_ptr`, ссылавшийся на этот объект), удаляем сам объект

- Идея: храним счётчик ссылок
- Когда `shared_ptr` копируется, увеличиваем счётчик
- Когда `shared_ptr` удаляется, уменьшаем счётчик
- Когда счётчик доходит до нуля (т.е. удалили последний `shared_ptr`, ссылавшийся на этот объект), удаляем сам объект
- **NB:** Не подходит, если есть циклические ссылки!

- Счётчик ссылок должен быть общим между всеми копиями `shared_ptr`, указывающими на один объект $\implies$ его придётся тоже хранить в куче

```
class shared_ptr {  
private:  
    struct storage {  
        std::size_t refcount = 0;  
        Object* pointer = nullptr;  
    };  
  
    storage* storage_;  
  
public:  
    ...  
};
```

- Настоящий `shared_ptr` устроен сложнее

- Настоящий `shared_ptr` устроен сложнее
- Позволяет передать произвольную функцию удаления (по умолчанию делает просто `delete`)

- Настоящий `shared_ptr` устроен сложнее
- Позволяет передать произвольную функцию удаления (по умолчанию делает просто `delete`)
- Позволяет ссылаться на часть объекта

- Настоящий `shared_ptr` устроен сложнее
- Позволяет передать произвольную функцию удаления (по умолчанию делает просто `delete`)
- Позволяет ссылаться на часть объекта
- Позволяет разбивать циклические ссылки или узнавать об удалении объекта через `weak_ptr`

- Одна из основных концепций ООП

- Одна из основных концепций ООП
- Хотим переиспользовать код классов в других классах

- Одна из основных концепций ООП
- Хотим переиспользовать код классов в других классах
- Вариант №1 – *композиция*: один класс является полем другого

- Одна из основных концепций ООП
- Хотим переиспользовать код классов в других классах
- Вариант №1 – *композиция*: один класс является полем другого
- Вариант №2 – *наследование*: один класс строится на базе другого

```
class Button {
private:
    const char* caption_;
    Action clickAction_;

public:
    Button(const char* caption)
        : caption_(caption)
    {}

    void draw() {
        drawRectangle(Color::Gray);
        drawText(caption_);
    }

    void onClick(Action clickAction) {
        clickAction_ = clickAction;
    }

    void click() {
        clickAction_.execute();
    }
};
```



```
class ColoredButton : public Button {
private:
    Color color_;

public:
    ColoredButton(const char* caption, Color color)
        : Button(caption) // вызов конструктора
        , color_(color)
    {}

    void draw() {
        drawRectangle(color_);
        // Ошибка: caption_ - private поле, об этом позже
        drawText(caption_);
    }

    void click() {
        color_ = Color::Red;
        // Вызов метода из базового класса
        Button::click();
    }
};
```

- `Button` – базовый класс (*base class*)

- `Button` – базовый класс (*base class*)
- `ColoredButton` – производный класс (*derived class*), или класс-наследник

- `Button` – базовый класс (*base class*)
- `ColoredButton` – производный класс (*derived class*), или класс-наследник
- В `ColoredButton` добавляются все поля и методы из базового класса

- `Button` – базовый класс (*base class*)
- `ColoredButton` – производный класс (*derived class*), или класс-наследник
- В `ColoredButton` добавляются все поля и методы из базового класса
- Конструктор класса-наследника должен вызвать конструктор базового класса (как и с полями, по умолчанию вызовется конструктор без аргументов)

- `Button` – базовый класс (*base class*)
- `ColoredButton` – производный класс (*derived class*), или класс-наследник
- В `ColoredButton` добавляются все поля и методы из базового класса
- Конструктор класса-наследника должен вызвать конструктор базового класса (как и с полями, по умолчанию вызовется конструктор без аргументов)
- Деструктор класса-наследника автоматически вызовет деструктор базового класса после того, как вызовет деструкторы всех полей класса-наследника

- `Button` – базовый класс (*base class*)
- `ColoredButton` – производный класс (*derived class*), или класс-наследник
- В `ColoredButton` добавляются все поля и методы из базового класса
- Конструктор класса-наследника должен вызвать конструктор базового класса (как и с полями, по умолчанию вызовется конструктор без аргументов)
- Деструктор класса-наследника автоматически вызовет деструктор базового класса после того, как вызовет деструкторы всех полей класса-наследника
- Указатель на класс-наследник автоматически приводится к типу указателя на базовый класс

- Хотим обрабатывать разные объекты одним и тем же кодом:

```
Button * buttons[3];
buttons[0] = new Button("Save");
buttons[1] = new Button("Load");
buttons[2] = new ColoredButton("Quit", Color::Blue);

void drawUI() {
    for (int i = 0; i < 3; ++i)
        buttons[i]->draw();
}
```


- Хотим обрабатывать разные объекты одним и тем же кодом:

```
Button * buttons[3];
buttons[0] = new Button("Save");
buttons[1] = new Button("Load");
buttons[2] = new ColoredButton("Quit", Color::Blue);

void drawUI() {
    for (int i = 0; i < 3; ++i)
        buttons[i]->draw();
}
```

- Функция **drawUI** не знает, какие классы на самом деле лежат по разным указателям, и всегда будет вызывать **Button::draw()** из базового класса

- Хотим обрабатывать разные объекты одним и тем же кодом:

```
Button * buttons[3];
buttons[0] = new Button("Save");
buttons[1] = new Button("Load");
buttons[2] = new ColoredButton("Quit", Color::Blue);

void drawUI() {
    for (int i = 0; i < 3; ++i)
        buttons[i]->draw();
}
```

- Функция **drawUI** не знает, какие классы на самом деле лежат по разным указателям, и всегда будет вызывать **Button::draw()** из базового класса
- Решение: *виртуальные методы*

```
class Button {  
public:  
    virtual void draw();  
    virtual void click();  
    ...  
};
```

```
class Button {  
public:  
    virtual void draw();  
    virtual void click();  
    ...  
};
```

- **virtual** говорит компилятору, что при вызове этого метода надо посмотреть, какой на самом деле класс лежит по этому указателю, и вызвать правильный метод

```
class Button {  
public:  
    virtual void draw();  
    virtual void click();  
    ...  
};
```

- **virtual** говорит компилятору, что при вызове этого метода надо посмотреть, какой на самом деле класс лежит по этому указателю, и вызвать правильный метод
- Теперь `buttons[2]->draw()` вызовет `ColoredButton::draw()`!

```
class Button {  
public:  
    virtual void draw();  
    virtual void click();  
    ...  
};
```

- **virtual** говорит компилятору, что при вызове этого метода надо посмотреть, какой на самом деле класс лежит по этому указателю, и вызвать правильный метод
- Теперь `buttons[2]->draw()` вызовет `ColoredButton::draw()`!
- Как это работает, обсудим в следующий раз

Виртуальный деструктор

```
Button * buttons[3];  
...  
for (int i = 0; i < 3; ++i)  
    delete buttons[i];
```

- В чём проблема?

Виртуальный деструктор

```
Button * buttons[3];  
...  
for (int i = 0; i < 3; ++i)  
    delete buttons[i];
```

- В чём проблема? `delete` видит указатель на `Button` и вызывает `Button::~~Button()`

Виртуальный деструктор

```
Button * buttons[3];  
...  
for (int i = 0; i < 3; ++i)  
    delete buttons[i];
```

- В чём проблема? `delete` видит указатель на `Button` и вызывает `Button::~~Button()`
- Правильный деструктор `ColoredButton::~~ColoredButton()` не будет вызван!

Виртуальный деструктор

```
Button * buttons[3];  
...  
for (int i = 0; i < 3; ++i)  
    delete buttons[i];
```

- В чём проблема? `delete` видит указатель на `Button` и вызывает `Button::~~Button()`
- Правильный деструктор `ColoredButton::~~ColoredButton()` не будет вызван!
- Решение: *виртуальный деструктор*

Виртуальный деструктор

```
class Button {  
public:  
    virtual ~Button();  
    ...  
};
```

Виртуальный деструктор

```
class Button {  
public:  
    virtual ~Button();  
    ...  
};
```

- Работает аналогично виртуальным методам: при вызове деструктора по указателю (в т.ч. как часть **delete**) компилятор проверит, какой тип на самом деле лежит по указателю, и вызовет правильный деструктор

Виртуальный деструктор

```
class Button {  
public:  
    virtual ~Button();  
    ...  
};
```

- Работает аналогично виртуальным методам: при вызове деструктора по указателю (в т.ч. как часть **delete**) компилятор проверит, какой тип на самом деле лежит по указателю, и вызовет правильный деструктор
- Хорошее правило: если в классе есть хотя бы один виртуальный метод, скорее всего деструктор тоже стоит сделать виртуальным

Protected уровень доступа

- Как решить проблему с тем, что `caption_` – приватное поле класса `Button`, и недоступно в `ColoredButton`?

Protected уровень доступа

- Как решить проблему с тем, что `caption_` – приватное поле класса `Button`, и недоступно в `ColoredButton`? Уровень доступа **protected**:

```
public Button {  
protected:  
    const char* caption_;  
    Action clickAction_;  
  
public:  
    ...  
};
```

Protected уровень доступа

- Как решить проблему с тем, что `caption_` – приватное поле класса `Button`, и недоступно в `ColoredButton`? Уровень доступа **protected**:

```
public Button {  
protected:  
    const char* caption_;  
    Action clickAction_;  
  
public:  
    ...  
};
```

- **protected** означает, что поле или метод недоступно снаружи класса, но **доступно** в его наследниках

Protected уровень доступа

- Как решить проблему с тем, что `caption_` – приватное поле класса `Button`, и недоступно в `ColoredButton`? Уровень доступа **protected**:

```
public Button {  
protected:  
    const char* caption_;  
    Action clickAction_;  
  
public:  
    ...  
};
```

- **protected** означает, что поле или метод недоступно снаружи класса, но **доступно** в его наследниках
- Полезно, если часть деталей реализации класса может быть нужна в наследниках

Protected уровень доступа

- Как решить проблему с тем, что `caption_` – приватное поле класса `Button`, и недоступно в `ColoredButton`? Уровень доступа **protected**:

```
public Button {  
protected:  
    const char* caption_;  
    Action clickAction_;  
  
public:  
    ...  
};
```

- **protected** означает, что поле или метод недоступно снаружи класса, но **доступно** в его наследниках
- Полезно, если часть деталей реализации класса может быть нужна в наследниках
- **NB:** В нашем конкретном случае можно было вместо **protected** поля сделать **public** метод `getCaption()`

- Посмотрим ещё раз на объявление ColoredButton:

```
class ColoredButton : public Button {  
    ...  
};
```

Protected и private наследование

- Посмотрим ещё раз на объявление `ColoredButton`:

```
class ColoredButton : public Button {  
    ...  
};
```

- Что означает `public Button`?

Protected и private наследование

- Посмотрим ещё раз на объявление `ColoredButton`:

```
class ColoredButton : public Button {  
    ...  
};
```

- Что означает `public Button`?
- Все неprivate поля и методы класса `Button` будут доступны в классе-наследники с такими же уровнями видимости

Protected и private наследование

- Посмотрим ещё раз на объявление `ColoredButton`:

```
class ColoredButton : public Button {  
    ...  
};
```

- Что означает `public Button`?
- Все неprivate поля и методы класса `Button` будут доступны в классе-наследнике с такими же уровнями видимости
- `protected Button` – аналогично, но `public` поля и методы становятся `protected` в наследнике

Protected и private наследование

- Посмотрим ещё раз на объявление `ColoredButton`:

```
class ColoredButton : public Button {  
    ...  
};
```

- Что означает `public Button`?
- Все неprivate поля и методы класса `Button` будут доступны в классе-наследнике с такими же уровнями видимости
- `protected Button` – аналогично, но `public` поля и методы становятся `protected` в наследнике
- `private Button` – все неprivate поля и методы становятся `private` в наследнике

- Не-**public** наследование нужно крайне редко, часто считается антипаттерном, и обычно его лучше заменить на композицию (т.е. добавить поле, а не отнаследоваться)

- Не-**public** наследование нужно крайне редко, часто считается антипаттерном, и обычно его лучше заменить на композицию (т.е. добавить поле, а не отнаследоваться)
- При не-**public** наследовании указатель на класс-наследник **не приводится** к указателю на базовый класс (потому что тогда модификаторы доступа становятся бесполезными)

- Не-**public** наследование нужно крайне редко, часто считается антипаттерном, и обычно его лучше заменить на композицию (т.е. добавить поле, а не отнаследоваться)
- При не-**public** наследовании указатель на класс-наследник **не приводится** к указателю на базовый класс (потому что тогда модификаторы доступа становятся бесполезными)
- Лучше не использовать не-**public** наследование без очень веской причины :)

Отличие class и struct

- Обычно ключевым словом **struct** описывают простые структуры, агрегирующие набор данных – без сложной логики, инициализации, деструкторов, и т.п.

Отличие class и struct

- Обычно ключевым словом **struct** описывают простые структуры, агрегирующие набор данных – без сложной логики, инициализации, деструкторов, и т.п.
- Ключевым словом **class** описывают сложные объекты с нетривиальной логикой

Отличие class и struct

- Обычно ключевым словом **struct** описывают простые структуры, агрегирующие набор данных – без сложной логики, инициализации, деструкторов, и т.п.
- Ключевым словом **class** описывают сложные объекты с нетривиальной логикой
- На самом деле, в C++ **class** и **struct** *почти* не отличаются: структуры тоже могут иметь методы, конструкторы, деструкторы, базовые классы, разграничение доступа, виртуальные методы, и т.д.

Отличие class и struct

- Обычно ключевым словом **struct** описывают простые структуры, агрегирующие набор данных – без сложной логики, инициализации, деструкторов, и т.п.
- Ключевым словом **class** описывают сложные объекты с нетривиальной логикой
- На самом деле, в C++ **class** и **struct** *почти* не отличаются: структуры тоже могут иметь методы, конструкторы, деструкторы, базовые классы, разграничение доступа, виртуальные методы, и т.д.
- Но отличия есть, и их ровно два: в уровне доступа по умолчанию и в виде наследования по умолчанию

Отличие class и struct

- Для `struct` уровень доступа и вид наследования по умолчанию – `public`

Отличие class и struct

- Для **struct** уровень доступа и вид наследования по умолчанию – **public**
- Другими словами, следующие два примера кода эквивалентны:

```
struct A : B {  
    Object x;  
};
```

```
struct A : public B {  
public:  
    Object x;  
};
```


Отличие class и struct

- Для `class` уровень доступа и вид наследования по умолчанию – `private`

Отличие class и struct

- Для **class** уровень доступа и вид наследования по умолчанию – **private**
- Другими словами, следующие два примера кода эквивалентны:

```
class A : B {  
    Object x;  
};
```

```
class A : private B {  
private:  
    Object x;  
};
```